

1.ÜNİTE - FLUTTER İLE MOBİL PROGRAMLAMAYA GİRİŞ

Mobil programlamanın temel kavramlarını, güncel pazar durumunu ve uygulama geliştirme yaklaşımlarını ele almaktadır. Ayrıca Google tarafından geliştirilen Flutter çerçevesinin avantajları ve kurulum süreçleri hakkında teknik bilgiler sunmaktadır.

Mobil Programlama Nedir ve Neden Gereklidir?

Teknolojinin gelişmesiyle birlikte akıllı telefonlar, sadece iletişim aracı olmaktan çıkıp veri işleme ve depolama kapasitesine sahip "cep bilgisayarlarına" dönüşmüştür. Günümüzde internet ve sosyal medya erişiminin %95'ten fazlası mobil platformlar üzerinden gerçekleşmektedir.

- **Donanım Farklılığı:** Bilgisayarlardan farklı olarak mobil cihazlar; GPS, ivmeölçer, jiroskop ve dokunmatik ekran gibi yoğun sensör kullanımı ve farklı giriş-çıkış birimleri içerir.
- **İşletim Sistemi Bağımlılığı:** Bir işletim sistemi için yazılan kod (örneğin Windows), başka bir sistemde (Linux veya MacOS) doğrudan çalışmaz; bu durum mobil tarafta Android ve iOS ayrımını doğurur.

Mobil Uygulama Geliştirme Türleri

Uygulama geliştirme süreçleri, projenin ihtiyaçlarına ve maliyetine göre üç ana başlıkta toplanır:

- **Doğal (Native):** İşletim sisteminin kendi diliyle (Android için Kotlin/Java, iOS için Swift) yazılır. En yüksek performansı ve cihaz özelliklerine tam erişimi sağlar ancak her platform için ayrı kod yazılması gerektiğinden maliyetlidir.
- **Çok Ortam Destekli (Cross-Platform):** Tek bir kod tabanıyla hem Android hem de iOS için uygulama geliştirilmesine olanak tanır. Flutter ve React Native bu kategoridedir.
- **Mobil Web:** Web teknolojileri (HTML, CSS, JS) kullanılarak tarayıcı tabanlı sunulan çözümlerdir. Hızlı geliştirilir ancak cihaz donanımına erişimi kısıtlıdır.

Mobil Programlama Dilleri ve Platformlar

Pazarda hakim olan diller ve bu dillerin kullanım amaçları şu şekildedir:

- **Android:** Temel diller Java ve daha modern olan Kotlin'dir. Google tarafından desteklenir.
- **iOS:** Apple ekosisteminde Objective-C'nin yerini alan Swift dili ve Xcode geliştirme ortamı kullanılır.
- **React Native:** JavaScript kullanarak işletim sisteminin arayüz bileşenlerini çağırır.
- **Flutter & Dart:** Google'ın geliştirdiği bu platform, arayüzü doğrudan **Skia** grafik motoruyla çizerek yüksek performans sunar.

Neden Flutter?

Flutter, modern uygulama geliştirme süreçlerinde sağladığı esneklik ve performans nedeniyle öne çıkmaktadır.

- **Geniş Destek:** Mobil cihazların yanı sıra web, masaüstü (Windows, Linux, MacOS) ve gömülü sistemler için çıktı verebilir.
- **Performans:** Ara katman kullanmak yerine grafik motoruyla doğrudan çizim yaptığı için doğal (native) uygulamalara yakın performans sergiler.
- **Widget Yapısı:** Arayüz tasarımı için kullanılan ve öğrenilmesi kolay olan zengin bileşen (Widget) kütüphanesine sahiptir.
- **Hızlı Geliştirme:** "Hot Reload" gibi özelliklerle kod değişikliklerini anında görme imkanı tanır.

Kurulum ve Gereksinimler

Flutter ile uygulama geliştirmeye başlamak için sistemin belirli yazılımsal ve donanımsal bileşenlere sahip olması gerekir.

- **Flutter SDK:** Uygulamanın temel geliştirme kiti. "Stable" (kararlı) kanalın kullanılması önerilir.
- **IDE (Geliştirme Ortamı):** VS Code veya Android Studio en yaygın tercihlerdir.
- **Android Toolchain:** Uygulamaları test etmek için Android SDK ve emülatör kurulumu zorunludur.

- **Komut Satırı:** "flutter doctor" komutu, sistemdeki eksik kurulumları tespit etmek için kullanılan en temel araçtır.

2.ÜNİTE - DART PROGRAMLAMA DİLİ TEMELLERİ

Flutter çerçevesinin temel taşı olan Dart dilinin yapısal özelliklerini ele almaktadır. Dart; Java, C# ve JavaScript gibi dillerin güçlü yanlarını birleştirerek modern, güvenli ve esnek bir programlama deneyimi sunar.

Dart Dilinin Önemli Özellikleri

Dart'ı diğer dillerden ayıran ve ona güç katan temel yapısal prensipler şunlardır:

- **Her Şey Bir Objedir:** Dart'ta sayılar, fonksiyonlar ve hatta null değeri bile bir objedir ve tümü `Object` sınıfından türetilir.
- **Güçlü Tip Tanımlama (Strongly Typed):** Dart tip güvenli bir dildir ancak `var` ile tip belirtmeden de değişken tanımlamaya izin verir; bu durumda tip, atanan ilk değerle sabitlenir.
- **Null Safety:** 2.12 sürümüyle gelen bu özellik, siz aksini belirtmedikçe (`?` operatörü ile) değişkenlerin null değer almasını engeller.
- **Erişim Belirteçleri:** `public`, `private` gibi anahtar kelimeler yoktur. Bir değişkenin veya fonksiyonun adının önüne alt çizgi (`_`) eklenmesi, onu bulunduğu kütüphaneye özel (private) yapar.

Değişkenler ve Sabitler

Değişken tanımlama ve bellek yönetimi Dart'ta oldukça esnektir:

- **var, final ve const:** Değişebilen veriler için `var`, bir kez değer atanıp değişmeyecek veriler için `final`, derleme zamanında değeri belli olan sabitler için `const` kullanılır.
- **Late Düzenleyicisi:** `late` anahtar kelimesi, null olamayacak bir değişkenin değerinin tanımlama anında değil, daha sonra (ilk kullanımdan önce) atanacağını belirtir.
- **Varsayılan Değerler:** `null-safety` kapalıysa her değişkenin varsayılan değeri `null`'dır; çünkü hepsi birer nesnedir.

Temel Veri Türleri

Dart, zengin bir yerleşik veri türü koleksiyonuna sahiptir:

- **Sayılar (int, double):** Tam sayılar için int, ondalıklı sayılar için double kullanılır.
- **String:** Metinler için kullanılır. "String Interpolation" (\$değişken veya \$ {işlem}) ile metin içine dinamik veri ekleme desteği çok güçlüdür.
- **Koleksiyonlar (List, Set, Map):**
 - **List:** Sıralı eleman dizileridir (indeks 0'dan başlar).
 - **Set:** Sıralanmamış, her elemandan sadece bir tane barındıran tekil kümelerdir.
 - **Map:** Anahtar-değer (key-value) çiftleri ile veri saklar.

Fonksiyonlar

Fonksiyonlar Dart'ta "birinci sınıf vatandaş" (first-class objects) olarak kabul edilir.

- **Esnek Tanımlama:** Geri dönüş tipi veya parametre tipi belirtmek zorunlu değildir; Dart bunları kullanım şeklinden çıkarabilir.
- **Ok Operatörü (=>):** Tek satırlık fonksiyonlar için gövde yerine kullanılır.
- **Parametre Çeşitleri:**
 - **İsimlendirilmiş Parametreler:** { } içinde tanımlanır, çağrılırken parametre_adi: değer şeklinde kullanılır.
 - **Opsiyonlu Parametreler:** [] içinde tanımlanır, zorunlu değildir.
- **Anonim Fonksiyonlar:** İsmi olmayan, sadece belirli bir işlevi yerine getiren fonksiyonlardır.

Operatörler ve Akış Kontrolü

Modern programlama ihtiyaçları için özel operatörler sunulur:

- **Tür Kontrolü:** as (tür dönüşümü), is (tür kontrolü).
- **Null Kontrolü:** ??= (değişken null ise ata), ?? (null ise alternatif değer kullan).

- **Kademeli İşlemler (..):** Aynı nesne üzerinde art arda işlem yapmayı sağlar.
- **Döngüler:** Klasik döngülerin yanı sıra listeler için `for-in` ve `forEach()` yapıları mevcuttur.

3.ÜNİTE - DART PROGRAMLAMA DİLİ GELİŞMİŞ ÖZELLİKLER

Dart'ın nesneye yönelimli programlama (OOP) yapısını, esnek miras alma modellerini ve I/O gecikmelerini önleyen asenkron çalışma prensiplerini kapsamaktadır. Dart, karmaşık uygulama yapılarını basitleştiren modern bir söz dizimi sunar.

Sınıf Yapısı ve Nesne Modeli

Dart, nesne tabanlı bir dildir ve her obje kendi sınıfının bir örneğidir.

- **Miras Alma (Inheritance):** Dart, tekli ana sınıf mirasını (`extends`) destekler ancak "mixin" tabanlı yapısıyla çoklu miras esnekliği sağlar.
- **Erişim Belirleyicileri:** `public`, `protected` veya `private` anahtar kelimeleri yoktur. İsimlendirmede kullanılan **alt çizgi (_)**, bir üyenin sadece o kütüphaneye özel (`private`) olduğunu belirtir.
- **Getter ve Setter:** Bir objenin özelliklerini hesaplayarak döndürmek veya değer atamak için kullanılır.

Yapıcı (Constructor) Fonksiyonlar

Dart, nesne oluşturma süreçlerini kolaylaştırmak için çeşitli yapıcı türleri sunar:

- **İsimlendirilmiş Yapıcılar:** Bir sınıftan farklı amaçlarla (örneğin JSON verisinden nesne üretmek için `.fromJson`) nesne üretilmesini sağlar.
- **Fabrika (Factory) Yapıcılar:** Her çağrıldığında yeni bir örnek oluşturmak yerine, mevcut bir örneği döndürebilir (önbellek kullanımı için idealdir).
- **this Kısayolu:** Yapıcı parametrelerini doğrudan üye değişkenlere aktarmak için kullanılır, böylece fonksiyon gövdesi yazma gereksinimi ortadan kalkar.

Miras ve Çok Biçimlilik (Polymorphism)

Dart'ta sınıflar arasındaki ilişkiler üç ana anahtar kelimeyle yönetilir:

- **extends:** Bir sınıfın tüm özellik ve davranışlarını devralmak ve genişletmek için kullanılır.
- **implements:** Bir sınıfı "arayüz" (interface) gibi kullanmayı sağlar; bu durumda üst sınıfın tüm fonksiyonları alt sınıfta yeniden tanımlanmalıdır.
- **with (Mixin):** Sınıflara, hiyerarşik bir miras bağı kurmadan ek özellikler ve yetenekler kazandırmak için kullanılır.

Kütüphaneler ve Genişletme Metotları

Kodun modülerliği ve tekrar kullanımı için kütüphane yapısı kritik öneme sahiptir:

- **import:** Harici paketleri veya kütüphaneleri projeye dahil eder. as ile takma isim verilebilir, show veya hide ile kütüphanenin sadece belirli kısımları alınabilir.
- **Genişleme Metotları (Extension Methods):** Müdahale edilemeyen (hazır) sınıflara yeni fonksiyonlar eklenmesini sağlar. Örneğin, String türüne özel bir `parseInt()` metodu bu yolla eklenebilir.

Jenerikler (Generics)

Jenerik tanımlamalar (`<T>`), kod tekrarını azaltır ve tip güvenliğini artırır.

- Örneğin, `List<String>` sadece metinlerden oluşan bir liste tanımlarken, jenerik bir sınıf farklı veri türleri için aynı mantıkla tekrar derlenebilir.

Asenkron Programlama

Dart, uzun süren işlemleri (dosya okuma, internetten veri çekme) uygulamayı dondurmadan yapmak için asenkron bir yapı kullanır.

- **Future ve Stream:** Sonradan gelecek tek bir değeri (Future) veya sürekli bir veri akışını (Stream) ifade eder.
- **async ve await:** Asenkron kodun, senkron kod gibi sıralı ve okunabilir yazılmasını sağlar.
- **Isolate:** Dart, geleneksel "multi-thread" yerine, her bir iş parçasığının kendi bellek bölgesine sahip olduğu "isolate" modelini kullanarak veri tutarsızlıklarını önler.

4.ÜNİTE - WIDGET KAVRAMINA GİRİŞ

Flutter projelerinin anatomisini ve kullanıcı arayüzünü (UI) oluşturan en temel yapı taşı olan "Widget" kavramını incelemektedir. Flutter'da ekranda gördüğünüz her şey bir widget'tır ve bu yapılar bir ağaç hiyerarşisi içinde düzenlenir.

Flutter Proje Yapısı

Yeni bir Flutter projesi oluşturulduğunda, belirli bir klasör düzeni standart olarak gelir:

- **android / ios:** Platforma özgü (native) ayarların ve izinlerin bulunduğu klasörler.
- **lib:** Uygulamanın asıl kodlarının yazıldığı yerdir. `main.dart` dosyası uygulamanın giriş noktasıdır.
- **pubspec.yaml:** Projenin bağımlılıklarının (kütüphaneler), varlıkların (resimler, fontlar) ve versiyon bilgilerinin yönetildiği yapılandırma dosyasıdır.

Widget Nedir?

Flutter'da widget'lar, uygulamanın görsel arayüzünü tanımlayan konfigürasyon nesneleridir. Bir buton, bir metin, bir hizalama bileşeni ve hatta uygulamanın kendisi birer widget'tır.

- **Widget Ağacı (Widget Tree):** Widget'lar iç içe geçerek bir hiyerarşi oluşturur. Üstteki widget (parent), altındaki widget'ları (children) yönetir.
- **Kompozisyon Yapısı:** Karmaşık arayüzler, basit widget'ların bir araya getirilmesiyle (composition) oluşturulur.

Stateless ve Stateful Widget Farkı

Flutter'da widget'lar, durum (state) yönetimine göre iki ana kategoriye ayrılır:

- **Stateless Widget (Durumsuz):** Ekran çizildikten sonra içeriği değişmeyen widget'lardır. Sabit bir başlık veya ikon gibi kullanıcı etkileşimiyle değişmeyecek yapılar için kullanılır.
- **Stateful Widget (Durumlu):** Kullanıcı etkileşimi (tıklama, veri girişi) veya zamanla değişebilen bir veriye sahip widget'lardır. İçeriğin güncellenmesi gerektiğinde `setState()` metodu çağrılarak arayüzün o kısmı yeniden çizilir.

Temel Arayüz Widget'ları

Uygulama geliştirirken en sık kullanılan bazı hazır bileşenler şunlardır:

- **MaterialApp & Scaffold:** Google'ın Material Design standartlarını uygulayan temel iskelet yapısıdır. Scaffold; uygulama çubuğu (AppBar), gövde (Body) ve alt menü gibi alanları hazır sunar.
- **Container:** Diğer widget'ları sarmalayan; kenar boşluğu (padding/margin), renk ve boyutlandırma gibi dekorasyon özellikleri sunan kutu modelidir.
- **Center:** İçindeki nesneyi ekrana göre tam merkezler.
- **Text:** Ekranda metin göstermeyi sağlar.

Kullanıcı Etkileşimlerini Yakalama

Kullanıcıların ekranla olan etkileşimini yönetmek için çeşitli araçlar kullanılır:

- **Düğmeler:** ElevatedButton, TextButton, IconButton gibi hazır yapılar onPressed parametresi ile tıklama olayını yakalar.
- **GestureDetector:** Hazır tıklama özelliği olmayan herhangi bir widget'a (örneğin bir resim) dokunma, çift tıklama veya uzun basma gibi yetenekler kazandırmak için kullanılır.

5.ÜNİTE - FLUTTER'DA ARAYÜZ DÜZENLEME

Flutter'da çoklu widget yapılarını hizalamak, paketlemek ve ekranın farklı bölgelerine yerleştirmek için kullanılan mizanpaj (layout) tekniklerini kapsamaktadır. Flutter'ın esnek yapısı, her türlü ekran boyutuna uygun tasarımlar yapmamıza olanak tanır.

Çoklu Widget Barındıran Yapılar

Tek bir çocuk (child) alan widget'ların aksine, birden fazla widget'ı liste olarak alan (children) yapılar arayüzün iskeletini oluşturur:

- **Row (Satır):** Widget'ları yatay düzlemde yan yana dizmek için kullanılır.
- **Column (Sütun):** Widget'ları dikey düzlemde alt alta dizmek için kullanılır.

- **ListView:** Ekranı sığmayan çok sayıda elemanı kaydırılabilir (scrollable) bir liste şeklinde sunar.
- **Stack:** Widget'ları birbiri üzerine (üst üste binecek şekilde) yerleştirmek için kullanılır.

Hizalama ve Boyutlandırma (Flex & Expanded)

Row ve Column içerisindeki elemanların nasıl yayılacağını kontrol etmek için "Esnek (Flex)" yapı kullanılır:

- **MainAxisAlignment:** Ana eksenindeki (Row için yatay, Column için dikey) hizalamayı belirler (Başlangıç, orta, son veya aralıklı).
- **CrossAxisAlignment:** Karşı eksenindeki hizalamayı belirler.
- **Expanded:** İçindeki widget'ın, bulunduğu satır veya sütundaki boş alanın tamamını kaplamasını sağlar.
- **Flexible:** Expanded'a benzer ancak widget'ın boş alanı kaplarken kendi içeriğinden daha fazla büyümemesine izin veren esneklik ayarları sunar.

Gelişmiş Düzenleme Widget'ları

Arayüzü daha detaylı ve estetik hale getirmek için kullanılan yardımcı yapılar şunlardır:

- **Padding:** Bir widget'ın çevresine iç boşluk eklemek için kullanılır.
- **SizedBox:** Widget'lar arasına sabit genişlik veya yükseklikte boşluk bırakmak ya da bir widget'ı belirli boyutlara zorlamak için kullanılır.
- **SingleChildScrollView:** Ekran içeriği fiziksel sınırlara sığmadığında (örneğin klavye açıldığında) içeriğin kaydırılabilmesini sağlayarak "taşma (overflow)" hatalarını önler.
- **Wrap:** Satıra veya sütuna sığmayan elemanların otomatik olarak bir alt satıra/sütuna geçmesini sağlar.

Tasarımda Esneklik: AspectRatio ve ConstrainedBox

- **AspectRatio:** Widget'ın belirli bir en-boy oranında (örneğin 16:9) kalmasını sağlar.
- **ConstrainedBox:** Bir widget'ın alabileceği minimum ve maksimum boyut sınırlarını (constraints) belirlemek için kullanılır.

Görsel Bileşenler: Image ve Icon

- **Image:** İnternette (Image.network) veya yerel dosyalardan (Image.asset) resim görüntülemek için kullanılır. Yerel resimler için pubspec.yaml dosyasında tanımlama yapılmalıdır.
- **Icon:** Flutter ile hazır gelen veya harici paketlerle eklenen ikon kütüphanelerini kullanmayı sağlar.

6.ÜNİTE - KULLANICI ETKİLEŞİMİ VE MATERYAL YÖNETİMİ

Kullanıcıdan veri alma yöntemlerini, girdi kontrollerini ve kullanıcıya işlem sonuçları hakkında nasıl geri bildirim verileceğini ele almaktadır. Flutter'ın sunduğu zengin materyal bileşenleri sayesinde profesyonel ve etkileşimli formlar oluşturmak mümkündür.

Kullanıcıdan Veri Alma: TextField ve TextFormField

Kullanıcıdan metin tabanlı veri almak için kullanılan temel bileşenlerdir:

- **TextField:** Temel metin giriş alanıdır; kullanıcıdan isim, şifre veya mesaj gibi verileri almak için kullanılır.
- **TextFormField:** Form widget'ı ile entegre çalışan, veri doğrulama (validation) ve kaydetme özelliklerine sahip daha gelişmiş bir giriş alanıdır.
- **InputDecoration:** Giriş alanlarına ipucu metni (hint), etiket (label), ikon ve kenarlık gibi görsel özellikler eklemek için kullanılır.
- **Controller:** TextEditingController aracılığıyla giriş alanındaki metin programatik olarak okunabilir veya değiştirilebilir.

Seçim Bileşenleri ve Kontroller

Metin girişi dışındaki etkileşimler için kullanılan standart Materyal bileşenleridir:

- **Checkbox:** Kullanıcıya "doğru/yanlış" veya "açık/kapalı" gibi iki seçenek sunar.
- **Radio:** Bir grup seçenek içerisinde sadece bir tanesinin seçilmesine izin verir.

- **Switch:** Ayarlar ekranlarında sıkça görülen, kaydırmalı açma-kapama düğmesidir.
- **DropDownButton:** Bir liste içerisinde seçim yapmak için açılır menü sunar.

Kullanıcı Bilgilendirme ve Geri Bildirim

Yapılan işlemler hakkında kullanıcıyı bilgilendirmek için üç ana yapı kullanılır:

- **SnackBar:** Ekranın altında kısa süreliğine beliren, işlem onayı veya basit uyarılar için kullanılan hafif bir bilgilendirme kutusudur.
- **AlertDialog:** Kullanıcının dikkatini odaklamasını gerektiren, onay veya seçim bekleyen orta ekranda açılan pencerelerdir.
- **BottomSheet:** Ekranın altından yukarı doğru açılan, daha fazla seçenek veya içerik sunan panellerdir.

Form Yönetimi ve Doğrulama (Validation)

Birden fazla girdi alanını tek bir grup olarak yönetmek için Form widget'ı kullanılır.

- **GlobalKey:** Formun durumuna (FormState) erişmek ve doğruluğunu kontrol etmek için bir anahtar kullanılır.
- **Validator:** Girdi alanına yazılan verinin uygun olup olmadığını (örneğin e-posta formatı, boş geçilemez alan) kontrol eden fonksiyonlardır.

Tarih ve Saat Seçiciler

Kullanıcının hatasız bir şekilde zaman verisi girebilmesi için hazır diyalog pencereleri sunulur:

- **showDatePicker:** Takvim üzerinden tarih seçilmesini sağlar.
- **showTimePicker:** Saat ve dakika seçimi için dijital veya analog bir saat arayüzü açar.

7.ÜNİTE - EKRANLAR ARASI NAVİGASYON VE YÖNLENDİRME

Flutter uygulamalarında kullanıcıyı farklı sayfalar arasında nasıl gezdireceğimizi, sayfalar arası veri aktarımını ve navigasyon geçmişinin nasıl yönetileceğini

kapsamaktadır. Flutter, sayfa geçişlerini bir yığın (stack) mantığıyla yöneten güçlü bir yapı sunar.

Navigasyon Temelleri: Navigator ve Route

Flutter'da her bir ekran "Route" olarak adlandırılır ve bu ekranların yönetimi "Navigator" bileşeni tarafından yapılır.

- **Navigator:** Ekranları bir yığın (stack) yapısında üst üste koyan veya en üsttekini çıkaran mekanizmadır.
- **Push:** Yeni bir ekranı yığının en üstüne ekleyerek kullanıcıyı o sayfaya yönlendirir.
- **Pop:** En üstteki ekranı yığından çıkararak kullanıcıyı bir önceki sayfaya geri döndürür.

Rotalama Yöntemleri

Navigasyonu gerçekleştirmek için iki temel yaklaşım kullanılır:

- **Doğrudan Navigasyon (MaterialPageRoute):** Gidilecek sayfanın o anki kod bloğu içerisinde anlık olarak oluşturulmasıdır. Küçük ölçekli uygulamalar için pratiktir.
- **İsmlendirilmiş Rotalar (Named Routes):** Sayfaların önceden bir isimle (örneğin: /ayarlar) MaterialApp içinde tanımlanmasıdır. Büyük projelerde yönetimi kolaylaştırır ve kodun okunabilirliğini artırır.

Sayfalar Arası Veri Aktarımı

Navigasyon sırasında sayfalar arasında bilgi paylaşımı hayati önem taşır:

- **Veri Gönderme:** Yeni sayfaya giderken sınıfın yapıcı fonksiyonu (constructor) üzerinden veya arguments parametresi ile veri iletilebilir.
- **Veri Bekleme:** Pop işlemi yapılırken geri dönen bir değer (örneğin: bir seçim sonucu) Navigator.pop(context, veri) şeklinde üst sayfaya aktarılabilir. Bu işlem genellikle async ve await yapısıyla asenkron olarak gerçekleştirilir.

Gelişmiş Navigasyon Yapıları

Kullanıcı deneyimini artırmak için sunulan diğer navigasyon bileşenleri şunlardır:

- **Drawer (Yan Menü):** Ekranın sol veya sağından açılan, uygulamanın ana bölümlerine hızlı erişim sağlayan menüdür.
- **BottomNavigationBar (Alt Menü):** Ekranın en altında sabit duran ve genellikle 3-5 ana kategori arasında geçiş yapmayı sağlayan menüdür.
- **TabBar ve TabView:** Sayfa içeriğini sekmeler halinde ayırarak yatay geçişlerle içeriği organize eder.

Navigasyon Geçmişini Temizleme

Bazı durumlarda (örneğin: başarılı bir giriş işleminden sonra) kullanıcının geri tuşuyla giriş sayfasına dönmesini istemeyiz. Bu gibi durumlarda `pushReplacement` veya `pushAndRemoveUntil` metodları kullanılarak navigasyon geçmişi düzenlenir veya tamamen temizlenir.

8.ÜNİTE - ANİMASYONLAR VE GELİŞMİŞ ARAYÜZ ÖZELLİKLERİ

Flutter içerisinde animasyonlar, uygulamaları daha kullanıcı dostu ve ilgi çekici hale getirmek için kullanılan tekniklerdir. Hazır bileşenlerin çoğunda kendiliğinden hareket efektleri bulunmakla birlikte, geliştiricilere karmaşık ve özel animasyonlar oluşturmak için geniş bir kütüphane sunulmaktadır.

FLUTTER'DA ANİMASYON OLUŞTURMA ÇEŞİTLERİ

Flutter'da animasyonlar zorluk seviyelerine ve uygulama yöntemlerine göre temel kategorilere ayrılmaktadır.

- **Implicit Animations (Kapalı Animasyonlar):** Flutter çerçevesinde gelen hazır widgetlar (`AnimatedContainer`, `AnimatedAlign` vb.) kullanılarak yapılan en basit yaklaşımdır.
- **Explicit Animations (Açık Animasyonlar):** Hazır widgetların yetersiz kaldığı durumlarda `AnimatedBuilder` veya `Transition` sınıfları ile kontrolün geliştiricide olduğu yöntemdir.

- **Low Level Animations (Alt Seviye Animasyonlar):** CustomPainter gibi sınıflar kullanılarak animasyonlardaki çizimlerin özel olarak tasarlandığı kategoridir.
- **Third-Party Frameworks:** Flutter'ın kendi kütüphanesi dışında (Lottie vb.) harici araçların kullanılmasıdır.

IMPLICIT (HAZIR) ANİMASYON BİLEŞENLERİ

Bu bileşenler, standart widget'ların "Animated" ön eki almış halleridir ve parametre değişimlerini otomatik olarak görsel bir geçişe dönüştürürler.

- **AnimatedContainer:** Boyut, renk ve hizalama gibi özelliklerin değişimini süre (duration) parametresine bağlı olarak animasyonla gerçekleştirir.
- **AnimatedAlign:** Altındaki bileşenlerin ekran üzerindeki konum değişimlerini yumuşak bir geçişle sağlar.
- **AnimatedDefaultTextStyle:** Metin stillerindeki (boyut, renk, ağırlık) değişimleri canlandırır.
- **Ortak Özellikler:** Tüm implicit widgetlar animasyon süresi için duration ve geçiş hızı eğrisi için curve parametrelerini kullanır.

ANİMASYON KÜTÜPHANESİ VE TEMEL BİLEŞENLER

Karmaşık animasyonlar oluşturmak için kullanılan Explicit Animations yönteminde bazı temel sınıflar ve kavramlar kritik öneme sahiptir.

- **AnimationController:** Animasyonun süresini, yönünü (ileri, geri, tekrar) ve çalışma durumunu yöneten ana kontrol birimidir.
- **TickerProvider (vsync):** Animasyonun ekran yenileme hızıyla senkronize olmasını sağlar ve ekran kapalıyken kaynak tüketimini önler.
- **Curves:** Animasyonun zaman içindeki hız değişimini (lineer, yavaş başlayıp hızlanan vb.) belirleyen şablonlardır.
- **Tweens:** Animasyonun hangi değer aralığında (örneğin 0 ile 360 derece arası veya kırmızıdan maviye) gerçekleşeceğini tanımlar.

GELİŞMİŞ ARAYÜZ ÖZELLİKLERİ: GESTURES VE SLIVERS

Kullanıcı etkileşimlerini anlamlandırmak ve performanslı arayüzler oluşturmak için gelişmiş sistemler kullanılır.

- **Gestures (Hareketler):** Dokunmatik ekrandaki tıklama (Tap), çift tıklama (Double Tap), uzun basma (Long Press) ve sürükleme (Drag) gibi fiziksel hareketlerin algılanmasını sağlayan sistemdir.
- **GestureDetector:** Ekrandaki fiziksel hareketleri yakalayıp fonksiyonlara dönüştüren temel widgettır.
- **Slivers:** Büyük listelerde ve kaydırılabilir alanlarda (SliverAppBar, SliverList) ekranın daha efektif kullanılmasını ve AppBar'ın kaydırma sırasında daralması gibi akıcı efektlerin zahmetsizce yapılmasını sağlar.

HERO ANİMASYONLARI

Sayfalar arası geçişlerde süreklilik hissi yaratmak için kullanılan özel bir modeldir.

- İki farklı sayfa arasındaki ortak bir bileşenin (örneğin bir ürün resmi) uçarak yer değiştirmesi efektini sağlar.
- Her iki sayfadaki Hero widget'ının aynı "tag" değerine sahip olması animasyonun eşleşmesi için zorunludur.

9.UNİTE - DURUM YÖNETİMİ

Flutter'da arayüzün oluşturulması, geleneksel yöntemlerden farklı olarak **bildirimsel (declarative)** bir bakış açısıyla gerçekleştirilir. Bu yaklaşımda arayüz, uygulamanın o anki durumunun bir fonksiyonu olarak tanımlanır ($\$UI = f(state)$). Veri değiştiğinde Flutter, build metodunu tekrar tetikleyerek arayüzü bu yeni duruma göre baştan çizer.

BİLDİRİMSEL DÜŞÜNME VE DURUM TÜRLERİ

Flutter'da durum yönetimi, verinin uygulama içerisindeki etki alanına göre iki ana kategoriye ayrılır:

Geçici (Ephemeral) Durum: Sadece tek bir widget içerisinde kalan ve uygulamanın diğer kısımlarını ilgilendirmeyen verilerdir.

- StatefulWidget ve setState() metodu ile yönetilir.

- Örnekler: Bir sayfa içindeki aktif sekme, bir animasyonun o anki karesi veya bir metin alanına girilen geçici veriler.

Uygulama Durumu (App State): Uygulamanın birden fazla noktasında paylaşılan veya kullanıcı oturumu boyunca saklanması gereken verilerdir.

- Örnekler: Alışveriş sepeti içeriği, kullanıcı oturum bilgileri, bildirimler veya tema tercihleri.
- Bu tür durumların yönetimi için Provider gibi gelişmiş mimariler kullanılır.

PROVIDER PAKETİ İLE DURUM YÖNETİMİ

Flutter ekibi tarafından önerilen Provider, uygulama seviyesindeki durum takibini kolaylaştıran bir kütüphanedir. Bu sistem üç temel bileşen üzerinde yükselir:

1. ChangeNotifier: Veri modelindeki değişiklikleri dinleyicilere haber veren basit bir sınıftır.

- Veri güncellendiğinde `notifyListeners()` metodu çağrılarak arayüzün yenilenmesi tetiklenir.

2. ChangeNotifierProvider: ChangeNotifier tarafından üretilen veriyi widget ağacındaki alt elemanlara ulaştıran widget'tır.

- Veriye ihtiyaç duyan tüm widget'ları kapsayacak şekilde ağacın üst noktalarında konumlandırılır.
- Birden fazla veri modeli varsa MultiProvider yapısı tercih edilir.

3. Consumer: Değişikliği izleyen ve veri güncellendiğinde sadece ilgili kısmın yeniden çizilmesini sağlayan bileşendir.

- Performans optimizasyonu için widget ağacında sadece değişecek olan noktaya (mümkün olduğunca aşağılara) yerleştirilmelidir.

PROVIDER.OF VE PERFORMANS

Bazı durumlarda veriyi sadece okumak veya bir metodu tetiklemek gerekir; arayüzün güncellenmesine ihtiyaç duyulmaz.

- Bu durumlarda Consumer yerine `Provider.of<T>(context, listen: false)` yapısı kullanılır.
- `listen: false` parametresi, verideki değişimlerin widget'ı gereksiz yere tekrar çizmesini (rebuild) engeller, böylece uygulama performansı korunur.

10.ÜNİTE - KALICI VERİ SAKLAMA

Mobil uygulamalarda veriler normal şartlarda RAM üzerinde geçici olarak tutulur. Uygulama kapandığında bu verilerin kaybolmaması için dosya sistemi, anahtar-değer çiftleri veya ilişkisel veri tabanları gibi kalıcı depolama çözümleri kullanılır. Flutter, her iki büyük mobil platformun (Android ve iOS) dosya sistemi mimarilerine uygun erişim yöntemleri sunar.

DOSYA İŞLEMLERİ

Cihazın yerel hafızasına doğrudan dosya kaydetmek ve bu dosyaları okumak için kullanılan temel yöntemdir.

- **Internal ve External Memory:** Android cihazlarda Internal Memory uygulamaya özeldir (SandBox/Kum Havuzu), External Memory ise kullanıcı dosyaları için ortak alandır.
- **Path Provider:** İşletim sistemleri arasındaki klasör yapısı farklılıklarını gidermek için kullanılan, doğru dosya yolunu (Cache veya AppData gibi) bulan yardımcı eklentidir.
- **Süreç:** Dosya işlemleri; doğru yolun bulunması, dosya referansı oluşturma, yazma ve okuma şeklinde dört ana adımdan oluşur.
- **Asenkron Yapı:** Dosya okuma ve yazma işlemleri asenkron (Future, async / await) olarak gerçekleştirilmelidir.

SHARED PREFERENCES

Küçük boyutlu verileri, bir harita (Map) yapısında olduğu gibi "anahtar-değer" (key-value) çiftleri şeklinde saklamak için kullanılan pratik bir çözümdür.

- **Desteklenen Türler:** Sadece int, double, bool, String ve StringList türlerini destekler.
- **Kullanım Amacı:** Kullanıcı ayarları, tema tercihleri veya basit sayaçlar gibi küçük veriler için idealdir.
- **SandBox:** Bu veriler her uygulamaya özeldir ve diğer uygulamalar tarafından erişilemez.
- **Yöntemler:** Veri yazmak için set, okumak için get ve silmek için remove fonksiyonları kullanılır.

SQLITE VE SQFLITE

Büyük, ilişkisel ve karmaşık sorgular gerektiren verilerin saklanması için akıllı cihazlarda kullanılan yerleşik veri tabanı sistemidir.

- **Özellikleri:** Sunucusuz (serverless) çalışır, ayar gerektirmez (zero-configuration) ve işlemleri atomik olarak gerçekleştirir (transactional).
- **Sqflite:** Flutter'da SQLite işlemleri için kullanılan temel eklentidir; tablo oluşturma, veri ekleme (insert), güncelleme (update) ve silme (delete) işlevlerini sağlar.
- **Veri Dönüşümü:** SQLite'dan gelen veriler genellikle `List<Map>` formatındadır ve kod tarafında model sınıflarına dönüştürülmelidir.
- **SQL Injection:** Parametrik sorgular (whereArgs kullanımı gibi) kullanılarak güvenlik açıkları engellenmelidir.

ORM YAKLAŞIMI VE FLOOR KÜTÜPHANESİ

Nesne-İlişkisel Eşleşme (ORM), veri tabanındaki tabloları doğrudan kod içerisindeki nesnelerle eşleştirerek manuel veri dönüşümü zahmetini ortadan kaldıran bir yöntemdir.

- **Entity:** Veri tabanındaki tabloları temsil eden basit sınıf tanımlamalarıdır. `@primaryKey` ve `@ColumnInfo` gibi işaretleyicilerle şemalar belirlenir.
- **DAO (Data Access Object):** Veri tabanı sorgularını (Select, Insert vb.) yöneten soyut sınıftır; SQL karmaşasını koddan ayırır.
- **Otomatik Kod Üretimi:** Floor kütüphanesi, `build_runner` aracılığıyla arka plandaki zorlu bağlantı kodlarını otomatik olarak üretir.
- **Avantajı:** Manuel dönüşüm hatalarını azaltır, kodun okunabilirliğini ve bakımını kolaylaştırır.

11.ÜNİTE - AĞ VE FIREBASE

Günümüzde mobil uygulamaların uzak sunucularla iletişim kurmadan çalışması nadir bir durumdur. Flutter ekosisteminde ağ işlemleri, arayüz çerçevesinden bağımsız olarak Dart programlama dili ile yürütülür ve tüm ağ operasyonları asenkron olarak gerçekleştirilir. Bu ünite, verilerin metin formunda serileştirilmesi, HTTP protokolü üzerinden veri iletimi ve Google'ın bulut tabanlı hizmetler bütünü olan Firebase platformu ele alınmaktadır.

JSON SERİLEŞTİRME (SERIALIZATION)

Bellekteki nesnelerin ağ üzerinden iletilebilmesi için metin tabanlı bir formata dönüştürülmesi işlemidir. Restful servisler için en yaygın format JSON'dır.

- **Manuel Serileştirme:** Küçük ölçekli projelerde `dart:convert` kütüphanesi kullanılarak `jsonDecode()` ve `jsonEncode()` fonksiyonları ile yapılır.
- **Model Sınıfı Yöntemi:** Hataları azaltmak için sınıf içerisinde `fromJson()` ve `toJson()` fonksiyonları tanımlanarak veriler işlenir.
- **Otomatik Kod Üretimi:** Büyük projelerde `json_serializable` ve `build_runner` paketleri kullanılarak serileştirme kodları otomatik olarak üretilir; bu yöntem çalışma zamanı hatalarını minimize eder.

HTTP PAKETİ İLE AĞ İŞLEMLERİ

HTTP protokolü üzerinden Restful API'ları sorgulamak için kullanılan en sade ve yaygın kütüphanedir.

- **İzinler:** Android platformunda ağ işlemleri için `AndroidManifest.xml` dosyasına internet erişim izni eklenmesi zorunludur.
- **İstek Fonksiyonları:** Paket; GET, POST, DELETE, PUT ve PATCH gibi standart HTTP taleplerini gönderen hazır fonksiyonlar sunar.
- **Asenkron Cevaplar:** İstekler `Future<http.Response>` tipinde değer döndürür ve gelen verinin işlenebilmesi için sunucunun durum kodu (örneğin 200 OK) kontrol edilmelidir.
- **FutureBuilder:** Sunucudan gelecek cevabı beklerken ekranın yükleme, hata veya başarı durumlarına göre otomatik güncellenmesini sağlayan özel bir widgettir.

FIREBASE VE FLUTTERFIRE

Firebase, uygulama geliştiricilerin sunucu yönetimi, veri tabanı tasarımı ve güvenlik gibi karmaşık arka plan (backend) işleriyle uğraşmadan hızlı uygulama geliştirmesini sağlayan bir hizmettir.

- **Firebase Servisleri:** Kimlik doğrulama, NoSQL veri tabanı, bulut fonksiyonları ve mesajlaşma gibi geniş bir hizmet yelpazesi sunar.
- **FlutterFire:** Firebase servislerinin Flutter uygulamalarına entegre edilmesini sağlayan resmi kütüphaneler topluluğuna verilen addır.

- **Başlatma:** Firebase özelliklerini kullanmadan önce `firebase_core` paketi eklenmeli ve uygulama ana dizininde `Flutter.initializeApp()` fonksiyonu asenkron olarak çağrılmalıdır.
- **Bağlantı Yapısı:** Firebase ile kurulan tüm iletişimler asenkron yapıdadır ve veriler Future objeleri üzerinden yönetilir.

12.ÜNİTE - PAKET KULLANIMI

Flutter ve Dart ekosisteminde uygulama geliştirme sürecini hızlandırmak amacıyla, daha önce geliştirilmiş hazır kod bloklarının (paket ve eklentilerin) projeye dahil edilmesi sürecini kapsar. Bu modüler yapı sayesinde geliştiriciler, temel işlevleri her seferinde baştan yazmak yerine topluluğun sunduğu güvenilir çözümleri kullanabilirler.

PAKET VE EKLENTİ KAVRAMLARI

Flutter ekosisteminde dış kaynaklı kodlar yapısal özelliklerine göre iki temel başlıkta sınıflandırılır.

- **Package (Paket):** En temel haliyle `pubspec . yml` dosyasına sahip, içinde Dart kodları, resim, video gibi veri dosyaları barındırabilen klasör yapılarıdır.
- **Plugin (Eklenti):** Paketlerin özel bir formudur; Dart kodlarının yanı sıra uygulamanın çalıştığı platformun (Android, iOS, Web) yerel dilleriyle (Java, Swift vb.) iletişim kurmasını sağlayan düşük seviyeli kodlamalar içerir.

PAKET YÖNETİMİ VE ERİŞİM

Uygulamada kullanılacak paketlerin bulunması ve projeye entegre edilmesi belirli standartlara bağlıdır.

- **Pub.dev:** Flutter ve Dart için paketlerin paylaşıldığı ana merkezdir; paketlerin yayımlanması için dokümantasyon ve örnek kullanım gibi katı kurallar uygulanır.
- **Flutter Favorites:** Yeni geliştiriciler için Google ekibi tarafından seçilmiş, en popüler ve yüksek kaliteli paketlerin yer aldığı özel bir listedir.
- **Pubspec.yaml:** Projenin paket gereksinimlerinin tanımlandığı ana konfigürasyon dosyasıdır; bağımlılıklar `dependencies` başlığı altına eklenir.

- **Pubspec.lock:** Projeye indirilen paketlerin kesin versiyon bilgilerini saklayarak, farklı geliştiricilerin aynı çalışma ortamına sahip olmasını sağlayan dosyadır.

VERSİYON KONTROLÜ VE ÇAKIŞMA ÇÖZÜMLERİ

Birden fazla paketin bir arada kullanıldığı projelerde versiyon uyumsuzluklarını yönetmek kritik bir öneme sahiptir.

- **Şapka (^) Sembolü:** Versiyon numarasının önüne eklenerek belirli bir sürüm aralığının (örneğin 5.4.0 ve üzeri ancak 6.0.0 altı) kullanılmasına izin verir; bu sayede pub sistemi çakışmaları otomatik çözer.
- **Dependency Overrides:** Belirli bir paketin spesifik bir versiyonunun kullanılması zorunluysa, bu başlık altında tanımlanarak diğer bağımlılıkların kısıtlamaları geçersiz kılınabilir.
- **Flutter Pub Upgrade:** Projedeki paketleri, pubspec.yaml dosyasındaki kısıtlamalar dahilinde izin verilen en güncel sürümlere yükseltmek için kullanılan komuttur.

ALTERNATİF PAKET KAYNAKLARI VE ÖZEL PAKETLER

Paketler her zaman merkezi sunucudan (pub.dev) gelmek zorunda değildir; farklı kaynaklardan da projeye dahil edilebilirler.

- **Yerel Yollar (Path):** Proje klasörü içindeki başka bir alt klasörde bulunan paketler path anahtarı ile tanımlanabilir.
- **Git Repoları:** Bir GitHub veya farklı bir git sunucusundaki paketler, doğrudan URL bilgisi verilerek projeye eklenebilir.
- **Kendi Paketini Oluşturma:** flutter create --template=package komutuyla standart bir şablon kullanılarak yeni paketler geliştirilebilir.
- **Birim Testler (Unit Tests):** Bir paketin pub.dev üzerinde yayımlanabilmesi için içindeki fonksiyonların test edildiği unit test bloklarının yazılması zorunludur.

13.ÜNİTE - TEST VE HATA AYIKLAMA

Modern mobil uygulama geliştirme sürecinde test ve hata ayıklama (debugging), uygulamanın kalitesini ve sürdürülebilirliğini garantileyen en kritik aşamalardır.

Flutter, geliştiricilere hem kod seviyesinde hata ayıklama hem de uygulamanın farklı katmanlarını kapsayan gelişmiş otomatik test araçları sunar.

FLUTTER HATA AYIKLAMA ARAÇLARI

Hata ayıklama süreci, kodun çalışma zamanı davranışlarını incelemek ve performans sorunlarını tespit etmek için kullanılan çeşitli araçları kapsar.

- **Dart DevTools:** Tarayıcı tabanlı çalışan; performans analizi, CPU kullanımı, ağ takibi ve bellek yönetimi gibi birçok aracı barındıran kapsamlı bir araç setidir.
- **Flutter Inspector:** Widget ağacını görselleştirerek ekran düzenindeki hataları tespit etmeye ve arayüz yapısını anlamaya yardımcı olur.
- **Log Kaydı Tutma:** `print()`, `debugPrint()` veya `dart:developer` paketindeki `log()` fonksiyonu ile akış takibi yapılır.
- **Durma Noktaları (Breakpoint):** Kod akışını belirli satırlarda durdurarak değişkenlerin durumunu incelemeyi sağlar; `debugger()` fonksiyonu ile koşullu durdurmalar yapılabilir.

FLUTTER DERLEME MODELLERİ

Flutter projeleri, kullanım amacına göre üç farklı modda derlenebilir:

- **Debug Modu:** Geliştirme aşamasında kullanılır; HotReload özelliği açıktır, hata ayıklama servisleri çalışır ancak uygulama daha yavaştır.
- **Release Modu:** Uygulamanın son kullanıcıya sunulacak halidir; performans en üst seviyeye çıkarılır, kod optimize edilir ve boyut küçültülür.
- **Profile Modu:** Uygulama performansını gerçekçi bir şekilde ölçmek için kullanılır; hata ayıklama özellikleri kapalı, performans izleme özellikleri açıktır.

OTOMATİK TEST SÜREÇLERİ

Uygulama büyüdükçe manuel testler yetersiz kalır; bu noktada üç ana seviyede otomatik test uygulanır:

- **Unit Test (Birim Test):** En alt seviye testtir; tek bir fonksiyon veya sınıfın tutarlılığını ölçer. `test` ve `expect` fonksiyonları kullanılır.
- **Widget Test:** Bir arayüz bileşeninin (widget) beklenen tepkiyi verip vermediğini ve widget ağacındaki varlığını kontrol eder.

- **Integration Test (Entegrasyon Testi):** Uygulamanın bir bütün olarak veya büyük bölümlerinin birbiriyle etkileşimini ve gerçek cihaz performansını test eder.

TEST YARDIMCI BİLEŞENLERİ

Test süreçlerini kolaylaştırmak ve izole etmek için kullanılan özel yapılar mevcuttur:

- **Mocking (Taklit Etme):** Web servisi veya veri tabanı gibi canlı kaynaklara bağımlılığı ortadan kaldırmak için Mockito paketi ile bu yapıların sahte örnekleri oluşturulur.
- **WidgetTester:** Test ortamında widget'larla etkileşim kurmayı ve onları oluşturmayı (pumpWidget) sağlar.
- **Finder ve Matcher:** Widget ağacı içinde belirli kriterlere göre arama yapmayı (find.text) ve bulunan sonucun doğruluğunu teyit etmeyi (findsOneWidget) sağlar.

14.ÜNİTE - PERFORMANS, OPTİMİZASYON VE DAĞITIM

Uygulama geliştirme sürecinin son aşaması, kullanıcı deneyimini en üst düzeye çıkarmak için performansı iyileştirmek, paket boyutunu optimize etmek ve uygulamayı hedef platformlarda dağıtımına hazır hale getirmektir. Flutter'da performans temel olarak hız, bellek yönetimi, uygulama boyutu ve enerji tüketimi ekseninde değerlendirilir.

UYGULAMA BOYUTUNUN ÖLÇÜLMESİ VE ANALİZİ

Uygulama boyutu, hem kurulum süresini hem de cihaz üzerindeki depolama alanını etkileyen kritik bir metriktir.

- **Hatalı Ölçümden Kaçınma:** Uygulama boyutu asla "Debug" modunda ölçülmemelidir; bu modda hata ayıklama verileri nedeniyle boyut normalden çok daha büyüktür.
- **Analiz Araçları:** --analyze-size parametresi kullanılarak terminal üzerinden veya DartDevTools içindeki "App Size Tool" aracılığıyla paket içeriğinin boyut dağılımı detaylıca incelenebilir.

- **Minimum Boyut:** Flutter uygulamaları, mimarisi gereği yaklaşık 6.5 MB - 6.9 MB civarında bir sarmalayıcı (wrapper) kod boyutuyla başlar.

UYGULAMA BOYUTUNUN AZALTILMASI

Dağıtım paketlerini (APK, App Bundle, IPA) küçültmek için belirli optimizasyon teknikleri uygulanır.

- **Hata Ayıklama Bilgilerini Ayırma:** Derleme sırasında `--split-debug-info` etiketi kullanılarak sembol dosyaları paketten ayrıştırılabilir.
- **Kaynak Optimizasyonu:** Kullanılmayan varlıklar (resim, müzik vb.) projeden çıkarılmalı; mevcut PNG ve JPEG dosyaları sıkıştırılarak kullanılmalıdır.
- **Kütüphane Yönetimi:** Üçüncü taraf kütüphanelerin sadece gerekli kısımlarının yüklenmesi sağlanmalıdır.

RENDER PERFORMANSININ İYİLEŞTİRİLMESİ

Flutter, Skia grafik motoru ile yüksek performans sunar ancak hatalı kodlama yaklaşımları akıcılığı bozabilir.

- **Build Metodu Verimliliği:** `build()` metotları içinde ağır işlemlerden (I/O, ağ sorgusu vb.) kaçınılmalı, bu işlemler asenkron olarak yapılmalıdır.
- **16ms Kısıtı:** 60 FPS akıcılık için her bir çerçevenin (frame) en geç 16 milisaniyede hazırlanması gerekir.
- **Lazy Loading:** `ListView.builder` gibi yapılar kullanılarak sadece ekranda görünen elemanların çizilmesi sağlanmalı, bu sayede bellek ve işlemci kullanımı optimize edilmelidir.
- **Özel Efekt Kullanımı:** `Opacity` gibi ağır işlemci gücü gerektiren widget'lar yerine, mümkünse doğrudan renk kodlarıyla saydamlık yönetimi tercih edilmelidir.

DEVTOOLS PERFORMANS GÖRÜNÜMÜ

DartDevTools, darboğazları tespit etmek için üç temel görsel bileşen sunar:

- **Flutter Frames Chart:** Her bir çerçevenin UI ve Render sürelerini bar grafik olarak gösterir.
- **Timeline Events Chart:** Uygulama içindeki tüm olayların (HTTP sorguları vb.) zamanlamasını yansıtır.

- **CPU Profiler:** İşlemci üzerinde yürütülen fonksiyonların Call Tree ve Flame Chart gibi gösterimlerle analiz edilmesini sağlar.

DAĞITIM VE GÜVENLİK

Uygulamayı mağazalara (Google Play, App Store) yüklemeyden önce platforma özgü hazırlıklar yapılmalıdır.

- **İzolatlar (Isolate):** Ağır hesaplama gerektiren işler için ana iş parçacığından (UI Thread) bağımsız çalışan "Isolate" yapıları kullanılarak arayüz donmaları engellenmelidir.
- **Kod Karıştırma (Obfuscating):** Tersine mühendisliği zorlaştırmak ve kod güvenliğini sağlamak için derleme sırasında `--obfuscate` etiketi kullanılmalıdır.
- **Platform Hazırlıkları:** Her platform için özel uygulama ikonları, açılış ekranları (splash screen) ve API anahtarları düzenlenmelidir.